
LYCOSA

Systems Architecture and Engineering Report

A source-grounded study of local AI orchestration,
retrieval and controlled cloud execution

Prepared for the Lycosa project

abdra7

18 September 2026 | Revision 1.0 | English edition

Independent academic-style report.
No institutional affiliation or endorsement is claimed.

Executive abstract

The system in one statement

Lycosa is a LAN-first AI operations platform that coordinates existing computers through a central control plane and a native desktop dashboard. Its engineering contribution is operational integration: hardware-aware placement, explicit local/cloud routing, document retrieval, persisted workflow state and human approval. It does not train a new foundation model, pool GPU memory across machines, or grant a language model unrestricted operating-system access. [L1-L4]

Evidence and maturity

This report reconstructs the implementation inspected on 18 September 2026, incorporating the Sprint 12 follow-up for OpenRouter and bounded Agent MCP/RAG. Source inspection is the authority for structural claims. Automated regression results and the recorded 7 September live checks are separate forms of evidence; neither proves production reliability or general answer accuracy. The successful free-model check returned 15 for 7 + 8 with provider-reported cost zero. This is a dated observation, not a price or availability guarantee. [L5]

Principal conclusion

The architecture is coherent for an experimental small-team or homelab deployment. The main unresolved concerns are distributed resource reservation, durable asynchronous execution, calibrated multilingual retrieval, end-to-end transport hardening and semantic validation of model answers. The report makes these gaps visible rather than converting roadmap intentions into implemented capabilities. All diagrams are analytical views of the inspected implementation, with selected abstractions explicitly identified.

Document control

Project owner and maintainer: abdra7. Report revision: 1.0. Languages: independent English and Arabic editions. Academic presentation: numbered sections, vector figures, traceable requirements and author-date references. This is an independent engineering report; it is not a Harvard University publication, accreditation or endorsement.

Contents and figure guide

01	Introduction	4
02	Background and motivation	5
03	Problem statement and objectives	6
04	Scope and limitations	7
05	Research Methodology	8
06	User Stories - Access and Execution	10
07	Functional Requirements - Control Plane	14
08	System architecture diagram	20
09	Data flow diagram	22
10	Use case diagram	24
11	System class diagram	26
12	System interaction diagram	28
13	Entity-relationship diagram	30
A	Appendix A Verification and risks	32
B	Appendix B Conclusions and roadmap	33
R	References and source map	34

READING THE DIAGRAMS

Figures 1-6 are full-page vector diagrams in Sections 8-13. Diagram pages use landscape orientation for legibility. Technical class, API and database identifiers are preserved across both editions.

01 Introduction

1.1 System purpose

Lycosa provides a common operating surface for heterogeneous AI-capable devices. A user submits a task through Flutter; FastAPI authenticates the request, records its lifecycle and either dispatches to an eligible local Agent or invokes an explicitly selected external provider. Knowledge and workflow services share this control plane. The useful unit of coordination is a task or workflow step, not a shard of a jointly trained model. [L2, L4]

1.2 Architectural vocabulary

A node is a registered device, an Agent is its execution service, and a runtime is a model-serving adapter such as Ollama. A role recommendation describes suitability; it is not proof that every role-specific tool exists. Usage denotes operational telemetry in the user interface. Legacy API and database names retain heartbeat terminology for compatibility. This distinction preserves the requested product language without falsely claiming a schema migration. [L2, L3]

1.3 Intended readership and contribution

The report serves maintainers, examiners and engineers evaluating whether the implementation meets a defensible scope. It links stakeholder needs to observable behavior and separates conceptual architecture from concrete relational storage. Its six diagrams answer different questions: deployment boundaries, information movement, actor goals, software contracts, execution order and data integrity. They should be read together rather than as interchangeable pictures.

02 Background and motivation

2.1 Fragmented local AI operations

Owning several computers does not automatically produce a manageable AI service. Models may be installed on one device but absent on another; available memory changes while an operator is choosing where to run a request. Separate command lines and independent document stores increase the cost of reproducing a result. Lycosa responds with inventory, role recommendations, routing explanations and a shared record of execution. Reduced operational friction is the design hypothesis; no controlled productivity study has yet established its magnitude. [L1-L3]

2.2 Retrieval and interoperability

Retrieval-augmented generation combines retrieved information with generation rather than assuming model parameters contain every relevant fact (Lewis et al., 2020). In Lycosa, document parsing, chunking, embeddings and vector search provide context to inference; provenance and refusal behavior remain engineering responsibilities. MCP defines host-client-server capability exchange (MCP, 2025). The project adopts a narrow stdio server with three tools, not an unrestricted agent network or an external MCP-client ecosystem. [L2, L3]

2.3 Local-first, not local-only

Local inference offers a deployment boundary under the operator's control, but hardware and model quality constrain it. Optional cloud routing offers a different capability envelope and a different data-disclosure boundary. The current OpenRouter path deliberately fixes one free NVIDIA model and caps provider prices at zero. This narrow policy makes cost intent auditable but does not promise unrestricted capacity, permanent free access or support for every vendor. General direct OpenAI, Gemini and DeepSeek integrations remain outside this implemented increment. [L2]

03 Problem statement and objectives

3.1 Problem statement

How can a small deployment coordinate heterogeneous local AI resources, relevant knowledge and selective external inference while preserving understandable routing, bounded authority and recoverable execution evidence? The problem is not merely invoking a model API. It includes identifying eligible devices, distinguishing a failed retrieval from missing evidence, preventing unauthorized cloud disclosure, and retaining enough state to explain a workflow decision. [L2, L3]

3.2 Engineering objectives

O1: make registration, liveness and role recommendation observable. O2: route tasks only to candidates satisfying model, freshness, privacy and resource constraints. O3: support collection-scoped retrieval and bounded evidence answering. O4: compose task, retrieval, parallel and approval steps with persisted outcomes. O5: separate provider credentials from user task payloads. O6: expose capabilities honestly through the dashboard and MCP. Each objective is traced to requirements and evaluation evidence in Sections 6-7 and Appendix A; objectives are not retrospective claims of complete production readiness.

3.3 Acceptance philosophy

Transport success, application success and answer quality are different criteria. HTTP 200 can carry an incomplete model response; a valid source identifier does not establish that the cited passage supports the claim. Acceptance therefore checks contracts, authorization and state transitions independently from content correctness. Proposed operational targets require a declared hardware profile and workload before measurement. This report does not invent throughput, latency percentiles, uptime or economic savings.

04 Scope and limitations

4.1 Implemented boundary

The inspected code includes Flutter desktop screens, a FastAPI controller, SQLAlchemy persistence, PostgreSQL, Qdrant, optional Redis coordination, Prometheus/Grafana, Python Agents, Ollama inference and explicit cloud adapters. Node onboarding, Usage reporting, task failover, knowledge ingestion and declarative workflows are implemented. The new Agent HTTP evidence endpoint and optional MCP stdio companion share bounded answering logic. They are not automatically substituted into the existing desktop RAG route. [L1-L6]

4.2 Deliberately excluded

The increment does not implement autonomous shell execution, unrestricted file tools, arbitrary external MCP connections, general model-provider discovery, model training or tensor-parallel multi-host inference. Consumer subscription sign-in is not implemented. The Anthropic API adapter is opt-in and its live acceptance remains deferred; its presence must not be described as tested subscription access. A role named vision or storage is an inventory/routing concept, not evidence of a complete vision or distributed-storage subsystem. [L2, L3]

4.3 Material limitations

Usage is a sampled observation rather than an atomic capacity reservation. GPU memory is not aggregated across cards. Workflow execution remains request-bound rather than backed by a durable job queue. Knowledge vectors are controller-hosted, despite an optional node association in the schema. The default hashing embedder tokenizes Latin alphanumeric text; Arabic-only retrieval quality is not established. The optional English embedding model also does not establish multilingual parity. [L2, L6]

4.4 Evidence cutoff

Repository review is current to the report date. Live-model and desktop observations quoted later belong to the recorded September 7 session and were not rerun merely for document production. Code present on disk, code pushed to GitHub and code loaded by an existing process are distinct states; deployment requires an explicit rebuild/restart plan.

05 Research Methodology

A code-first design-science approach

This report treats Lycosa as an engineering artefact investigated through a design-science lens: identify a practical coordination problem, construct a software response, and evaluate its observable properties. The object of analysis is the repository snapshot of 18 September 2026, not an imagined production deployment. The study is descriptive and analytical rather than a controlled experiment. It does not claim interviews, a participant cohort, an institutional review, or statistically generalisable user outcomes. Architectural interpretation is grounded in executable interfaces, domain models, service boundaries and deployment configuration, with project documentation used as supporting context rather than unquestioned proof. [L1-L6]

Evidence collection and triangulation

Evidence is collected from six internal source groups: documentation, controller services and models, agent implementation, desktop implementation, automated tests, and infrastructure. A capability enters the implemented scope only when a corresponding execution path can be identified. Tests strengthen confidence in specific contracts, but test presence alone is not a successful execution result. Where documentation and code differ, the narrower demonstrable behaviour governs this report. Diagram elements are cross-checked against persistence models and request flow; conceptual simplification is explicitly separated from an exact deployment inventory. This makes the method reproducible without inventing stakeholder research. [L1-L6]

05 Research Methodology (continued)

Evaluation strategy and acceptance evidence

Evaluation is organised in layers. Unit tests address schemas, routing decisions, evidence budgets and credential rules; integration tests exercise persistence, workflow transitions and protocol boundaries. Optional live smoke tests use synthetic content to distinguish transport success from answer quality, and desktop inspection checks presentation separately from backend correctness. A supported evidence question, an unsupported question, privacy rejection, missing credentials and a rejected approval gate form a minimum negative-and-positive test set. Proposed acceptance criteria below are deliberately falsifiable. An individual successful model response establishes that scenario only; it does not demonstrate universal factual grounding, provider availability or usability. [L2-L5]

Validity, ethics and reproducibility

Internal validity is limited by mocked dependencies and implementation-coupled tests. External validity is limited by hardware, model size, network conditions and a small set of synthetic scenarios. Accordingly, performance thresholds are future acceptance targets unless a dated run documents conditions and measurements. Reproduction should record commit identifier, dependency versions, model identifier, resource limits, test command and outcome. Provider secrets must never enter the report or repository; real private documents must not be sent to external inference services merely to demonstrate the architecture. Outstanding discrepancies become explicit limitations or backlog items, not silently upgraded claims. [L3, L5, L6]

06 User Stories - Access and Execution

Scenario-based specification

These stories translate repository-observed roles and journeys into testable expectations; they are analyst-derived, not quotations from a user study. Acceptance clauses specify observable outcomes rather than internal implementation choices. The administrator governs access and provider configuration, while authorised operators submit work and inspect results. Implementation status describes available code paths, not a claim that every desktop scenario has been retested for this report. [L2, L4, L5]

US01 · Establish an authorised session

As an operator, I want to select my controller profile and authenticate so that I can manage the intended environment. Given a reachable controller and valid account, when I sign in, then the dashboard exposes only operations permitted by the server-side role. Given an invalid credential or expired session, when a protected request is made, then access is denied without disclosing credential values. Signing out must invalidate the applicable application session; it does not remove a controller-wide provider credential. Status: implemented authentication and profile paths; explicit role checks remain authoritative. [L2, L4]

US02 · Enrol and inspect a device

As an administrator, I want to issue a node credential and inspect a joining agent so that device participation is deliberate. Given a valid node key, when an agent registers and sends Usage telemetry, then the node inventory exposes identity, hardware and availability with a role recommendation. Given only an mDNS discovery announcement, when the device is discovered, then it is not automatically trusted or enrolled. Revoking its key must deny subsequent authenticated access. Status: implemented enrolment, advisory discovery and telemetry; role labels alone do not prove corresponding specialist tools exist. [L2-L4]

06 User Stories - Access and Execution (continued)

US03 · Run a local inference task

As an operator, I want to submit a prompt to an eligible local model and understand the outcome. Given an online dispatchable node with the requested model, when I submit an Ollama task, then the controller selects a compatible candidate and persists the result or failure. Given a candidate execution failure, then the recorded attempt trace explains the attempted node path. If no candidate qualifies, the task must fail visibly rather than pretend success. Status: implemented routing and task history; synchronous request execution is not a durable background job queue. [L2, L4]

US04 · Opt into constrained cloud execution

As an operator, I want an explicitly configured free-model route when local compute is unavailable. Given an administrator-configured OpenRouter credential, when I select the supported NVIDIA model and submit non-private synthetic input, then controller-side execution records output and numeric usage. Given `requires_privacy=true`, then external execution is rejected before sending inference content. A missing credential, provider rejection or unavailable free route must remain visible, without an unrequested paid fallback. Status: implemented exact-model path; configured credentials are not evidence of remote validity. [L2, L4]

06 User Stories - Knowledge and Coordination

Knowledge is an explicit trust boundary

The following stories distinguish collection retrieval, task-context injection and the agent's separate evidence-answering contract. These are related capabilities, but they are not interchangeable interfaces. Source identifiers support inspection; they are not a formal proof that an answer follows from a passage. Human approval governs workflow continuation rather than guaranteeing the truth or safety of every generated statement. [L2, L3]

US05 · Build and query a knowledge collection

As a knowledge operator, I want to upload a supported document and query a named collection so that retrieval is inspectable and scoped. Given a collection and a valid synthetic document, when ingestion succeeds, then document metadata and searchable vector chunks are available. When I retrieve a known phrase within that collection, then results include source, collection and relevance score. Given an unknown collection name, then an explicit error is returned rather than silently broadening the search. Status: implemented controller-managed ingestion and retrieval; fully distributed multi-node vector federation is outside this implementation. [L2, L4]

US06 · Ask an evidence-bounded local agent

As a developer integrating the agent, I want to supply retrieved passages and receive an answer with inspectable citations. Given an installed local model and relevant passages, when I call /rag/answer or the optional MCP answer_from_context tool, then the response contains source mapping, citation status and truncation information. Given no surviving evidence after filtering, then the agent refuses without model generation. Unknown citation IDs must be flagged, not manufactured into valid sources. Status: implemented bounded contract; the desktop's existing controller RAG path is not automatically migrated to this endpoint. [L3]

06 User Stories - Knowledge and Coordination (continued)

US07 · Control a multi-step workflow

As an operator, I want retrieval and task steps to share context while pausing for a human decision. Given a valid workflow containing an approval step, when execution reaches that step, then the run is persisted as paused and the approval step as pending_approval; downstream steps do not execute. When an authorised operator approves, then execution resumes; when rejected, then the run records failure without executing the blocked continuation. Retry attempts must remain inspectable. Status: implemented workflow state transitions, conditions and bounded task fan-out; no autonomous agent planning or crash-proof distributed workflow guarantee is implied. [L2, L4, L5]

US08 · Diagnose execution and availability

As an administrator, I want task history, node Usage and audit events to explain an operational failure. Given a failed task or stale node, when I inspect the dashboard and authorised API records, then I can distinguish unavailable capacity, rejected policy and runtime error where recorded. Provider credentials must remain absent from visible history and audit details. Status: implemented operational surfaces; a node shown offline still requires runtime investigation. Proposed usability acceptance: a new operator completes this diagnosis using the report's synthetic fixture without editing a database or reading a secret. [L2, L4-L6]

07 Functional Requirements - Control Plane

Status and acceptance convention

The requirements below restate the current implementation as verifiable obligations. Implemented means a source-level execution path exists; proposed acceptance means the criterion is a recommended release gate, not an already measured outcome. This distinction prevents design intent from being misreported as demonstrated reliability. Requirement identifiers are report-local and should be mapped to issue tracking before formal baselining. [L1, L5]

FR01 · Authentication and authorisation

The controller shall authenticate supported principals and enforce endpoint roles independently of desktop presentation. Status: implemented account sessions, API-key handling and role checks. Proposed acceptance: exercise valid, invalid, expired and revoked credentials against every protected route category; all unauthorised operations must be denied and authorised operations must retain their intended access. Administrative provider management must reject operator and node principals. Test records must contain credential references only, never reusable secret values. [L2, L5]

FR02 · Node enrolment and Usage

The platform shall enrol authenticated agents, retain hardware and runtime metadata, and derive operational availability from reported Usage freshness. Status: implemented registration, telemetry and advisory discovery. Proposed acceptance: register a synthetic node, update its metrics, advance time beyond the configured stale threshold, and verify the expected availability transition. Repeat registration must follow the documented identity contract without granting an unrelated key control of an existing node. Discovery alone must never establish dispatch authority. [L2, L3, L5]

07 Functional Requirements - Control Plane (continued)

FR03 · Explainable candidate selection

Local tasks shall be routed using dispatchability, role compatibility, installed models and available resource evidence, subject to requested privacy and memory constraints. Status: implemented heuristic routing; no global GPU reservation is claimed. Proposed acceptance: use a deterministic candidate fixture covering stale telemetry, absent models, insufficient RAM or VRAM and explicit CPU fallback. Each selected candidate must satisfy the enforced filters, and the routing explanation must identify the applied considerations. Removing all eligible candidates must produce a visible failure. [L2, L5]

FR04 · Persistent task outcomes

Task submission shall preserve input metadata, state transitions and result or error, with local execution attempts linked to participating nodes. Status: implemented synchronous dispatch and persisted history; direct OpenRouter tasks need no node execution row. Proposed acceptance: verify success, exhausted local failover and client-disconnect scenarios. Each terminal record must have the appropriate completion state and recoverable detail. A disconnected HTTP client must not by itself cancel shielded execution, but process termination is outside that guarantee. [L2, L5]

FR05 · Controlled provider credentials

Administrators shall explicitly manage controller-owned provider credentials through supported secure-vault or restricted session storage. Status: implemented; OpenRouter session storage requires one worker and disappears on restart. Proposed acceptance: verify role denial, invalid-value rejection, storage failure and successful removal of the selected storage entry. Status responses must reveal configuration only. The desktop must refuse credential submission over remote plaintext HTTP and redirects; deployment TLS remains an operator responsibility rather than a server-enforced universal transport guarantee. [L2, L4, L5]

07 Functional Requirements - Knowledge and Workflows

FR06 · Document lifecycle and retrieval

The knowledge service shall manage collections, extract supported document content, generate embeddings and retrieve ranked chunks with provenance. Status: implemented controller-managed relational metadata and Qdrant vectors. Proposed acceptance: ingest a deterministic fixture, verify searchable chunks and source identifiers, retrieve within a named collection, and test collection deletion, including its documents, embedding jobs and vectors across metadata and vector storage. Unknown collections must fail explicitly. Cross-collection retrieval must be described as merged controller-managed search, not distributed vector federation. Limits and supported formats must be taken from deployed configuration and loaders. [L2, L5]

FR07 · Explicit retrieval-to-generation policy

Controller task and workflow routes shall pass the specified knowledge query and collection into retrieval before contextual generation. Status: implemented legacy context injection, with fail-closed retrieval errors on the external OpenRouter route. Proposed acceptance: assert that a scoped fixture reaches the intended generation request, an unknown collection prevents external inference, and requires_privacy blocks the cloud path. An empty valid retrieval and a retrieval exception are distinct cases and must be evaluated separately. This requirement does not assert that legacy controller results expose the agent's structured citation contract. [L2, L5]

07 Functional Requirements - Knowledge and Workflows (continued)

FR08 · Bounded evidence answering

The agent shall accept at most 20 supplied evidence chunks, filter and deduplicate them, apply the requested character budget and expose citation diagnostics. Status: implemented local /rag/answer and shared MCP logic; retrieval remains caller-owned. Proposed acceptance: exercise empty evidence, filtered evidence, clipping, missing models, unknown citation IDs, refusal and runtime timeout. Inputs exceeding schema limits must be rejected before inference. Supported and unsupported fact fixtures must evaluate answer content independently of citation validity. Source-ID membership alone shall never be labelled factual-entailment verification. [L3, L5]

FR09 · Governed workflow execution

The workflow engine shall validate definitions, propagate declared step context, persist run and attempt state, and enforce approval barriers. Status: implemented sequential execution, conditions, retries, parallel task substeps and approval continuation. Proposed acceptance: cover a successful run, a failing retried task, a skipped condition, parallel branch failure, approval and rejection. No downstream operation may execute before an approval decision. Rejected runs must remain inspectable, with no accidental re-entry into the blocked continuation. Durable restart recovery, distributed locking and an asynchronous queue require separate design and verification. [L2, L5]

FR10 · Constrained MCP interoperability

The optional agent companion shall expose capability inspection, local model enumeration and evidence answering through stdio MCP. Status: implemented three-tool server, launched as a separate local process. Proposed acceptance: complete a real client handshake, enumerate exactly the supported tools, invoke valid requests, reject invalid schemas and reject an unknown shell-execution tool. The companion must close its runtime client on shutdown. It shall not silently gain arbitrary filesystem access, external MCP connections, autonomous tool loops or the registered service's Usage identity. Those capabilities remain outside scope rather than hidden extensions. [L3, L5]

07 Non-Functional Requirements and Release Gates

NFR01 · Security

Status: role checks, agent tokens and secret redaction are implemented; comprehensive security assurance remains proposed. Acceptance target: zero successful unauthorised operations in the route-role matrix and zero credential values in captured application logs, persisted task output and audit fixtures. Test hostile redirects and runtime errors. Internet exposure requires a separate deployment hardening review. [L2-L6]

NFR02 · Privacy and data minimisation

Status: explicit privacy gating and restricted provider-output persistence are implemented. Acceptance target: every external-provider privacy fixture produces zero outbound inference calls; persisted provider metadata contains only the intended numeric usage and routing fields. Proposed production work: document retention periods, deletion responsibilities and controller-wide credential-sharing consent. No tenant-isolation certification is claimed. [L2, L5]

NFR03 · Predictable bounded execution

Status: agent model-list and generation deadlines are 5 and 90 seconds; grounded HTTP and MCP calls each have a two-request concurrency limit. Acceptance target: use controlled slow adapters to verify deadlines, third-request rejection and counter release after cancellation. Proposed benchmark: report p50 and p95 separately for retrieval and inference on declared hardware; no latency achievement is asserted. [L3, L5]

NFR04 · Reliability and recovery

Status: recorded attempts, local failover and persisted workflow states exist. Acceptance target: fault-injection fixtures end in a truthful terminal or approval state with attributable error detail. Proposed production gate: demonstrate backup restoration and document recovery objectives. Client-disconnect shielding must not be presented as recovery from controller process loss, power failure or an unavailable database. [L2, L5, L6]

07 Non-Functional Requirements and Release Gates (continued)

NFR05 · Observability and auditability

Status: task history, Usage telemetry, audit records, metrics and dashboard events are implemented. Acceptance target: each test task and workflow can be traced by identifier from submission to stored outcome, and metric counters reflect controlled success/failure fixtures. Proposed alert objectives must specify thresholds and recipients before operation; event delivery alone is not durable forensic retention. [L2, L4-L6]

NFR06 · Usability and accessibility

Status: native desktop screens and widget tests exist; formal usability and accessibility studies are not established. Proposed acceptance: eight story walkthroughs complete without database edits; keyboard focus reaches every actionable control; errors distinguish policy, configuration and execution failure. Test readable layouts at 100% and 150% scaling. Provider-selection persistence remains an explicit behaviour to verify. [L4, L5]

NFR07 · Maintainability and portability

Status: controller, agent and Flutter desktop are separated and covered by targeted tests. Proposed acceptance: a clean environment reproduces dependency installation, linting and applicable tests using documented commands. Record skipped optional-MCP tests explicitly. Platform support requires a build-and-launch result for each claimed operating system; one Windows run does not validate all desktop targets. [L1, L3-L6]

NFR08 · Grounding quality and uncertainty

Status: evidence limits, refusal handling and source-ID diagnostics exist, not semantic verification. Proposed evaluation: pre-register at least 30 supported, unsupported and injection-bearing synthetic cases; independently score factual support, citation correctness and appropriate abstention. Publish failures and model configuration. Any numerical release threshold should be agreed before evaluation, not selected retrospectively to make a model appear reliable. [L3, L5]

08 System architecture diagram

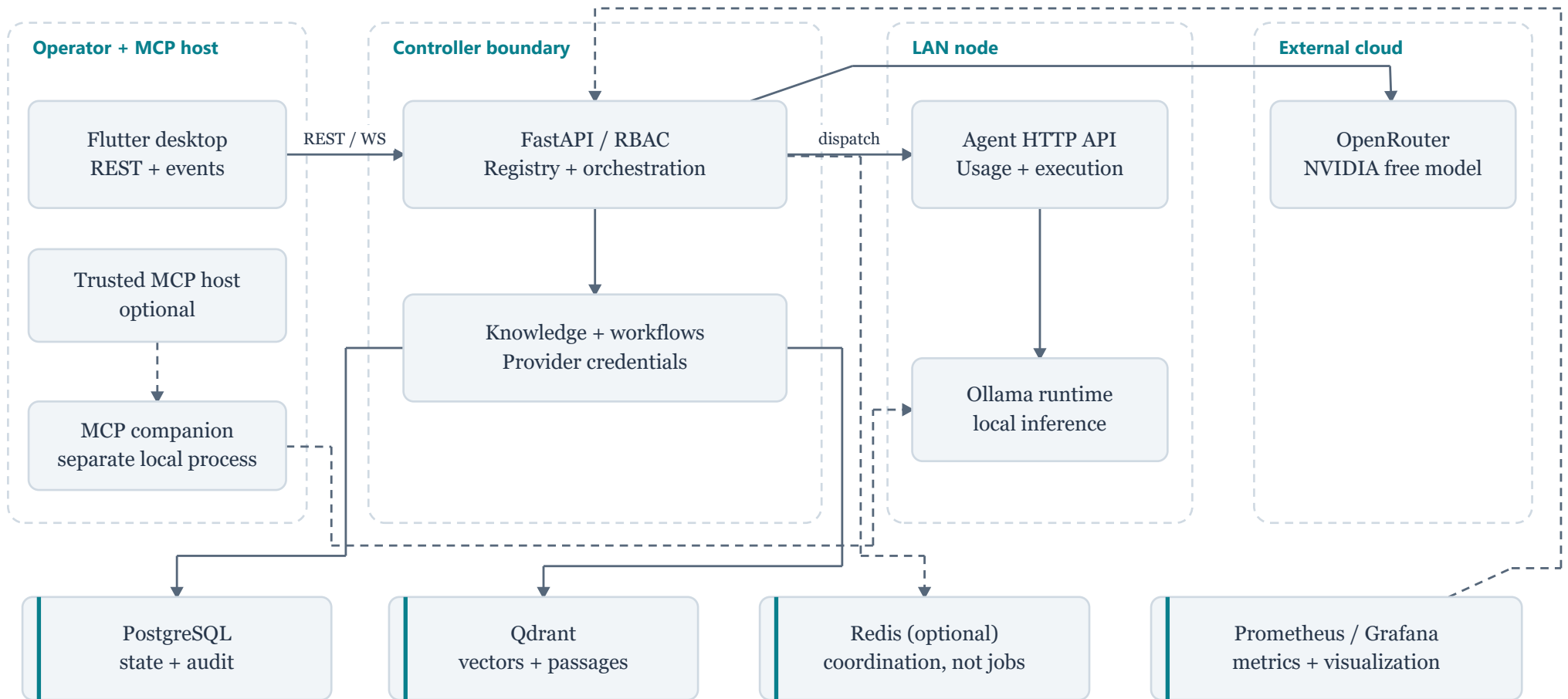


Figure 1. Deployment and trust boundaries

Solid arrows denote implemented communication paths. Dashed links indicate an optional companion or opt-in path. Storage and observability are controller-side. OpenRouter bypasses node scheduling; Ollama inference uses the Agent route. No arrow represents shared GPU memory or an autonomous external-tool loop. This is an analytical deployment view derived from controller, Agent and Compose code. [L2, L3, L6]

08 Architectural decisions and trade-offs

8.1 Control plane and execution plane

The controller owns identity, scheduling, knowledge routing and durable operational records; Agents own device-local execution. This separation makes model inventory and hardware variability manageable without distributing every business rule. It also makes the controller a critical dependency. PostgreSQL availability, schema migration and backup restoration must be treated as system-level concerns, not optional deployment polish. The local Agent token protects execution requests but is not a substitute for authenticated, encrypted network transport. [L2, L3, L6]

8.2 Worker scaling and secret lifetime

Redis can relay events, share rate-limit/login windows and coordinate leader-gated work across controller workers. It is not the durable workflow queue. Process memory is not shared automatically across workers (FastAPI, n.d.). Therefore the explicit OpenRouter session credential store requires a single controller worker and is lost on restart. A provisioned OS vault is the durable alternative. A host Windows vault does not automatically become a usable secret service inside a Linux container. [L2, L6]

8.3 Failure-domain interpretation

Node failover can improve the chance of task completion when another eligible local device exists, but it cannot compensate for a failed control-plane database. A failed external provider should return a sanitized failure rather than silently select a paid provider. Prometheus and Grafana support diagnosis; they do not create an availability guarantee. Recommended next steps are tested restoration, explicit transport policy and bounded asynchronous admission, each accompanied by measurable acceptance criteria rather than a claim of automatic resilience.

09 Data flow diagram

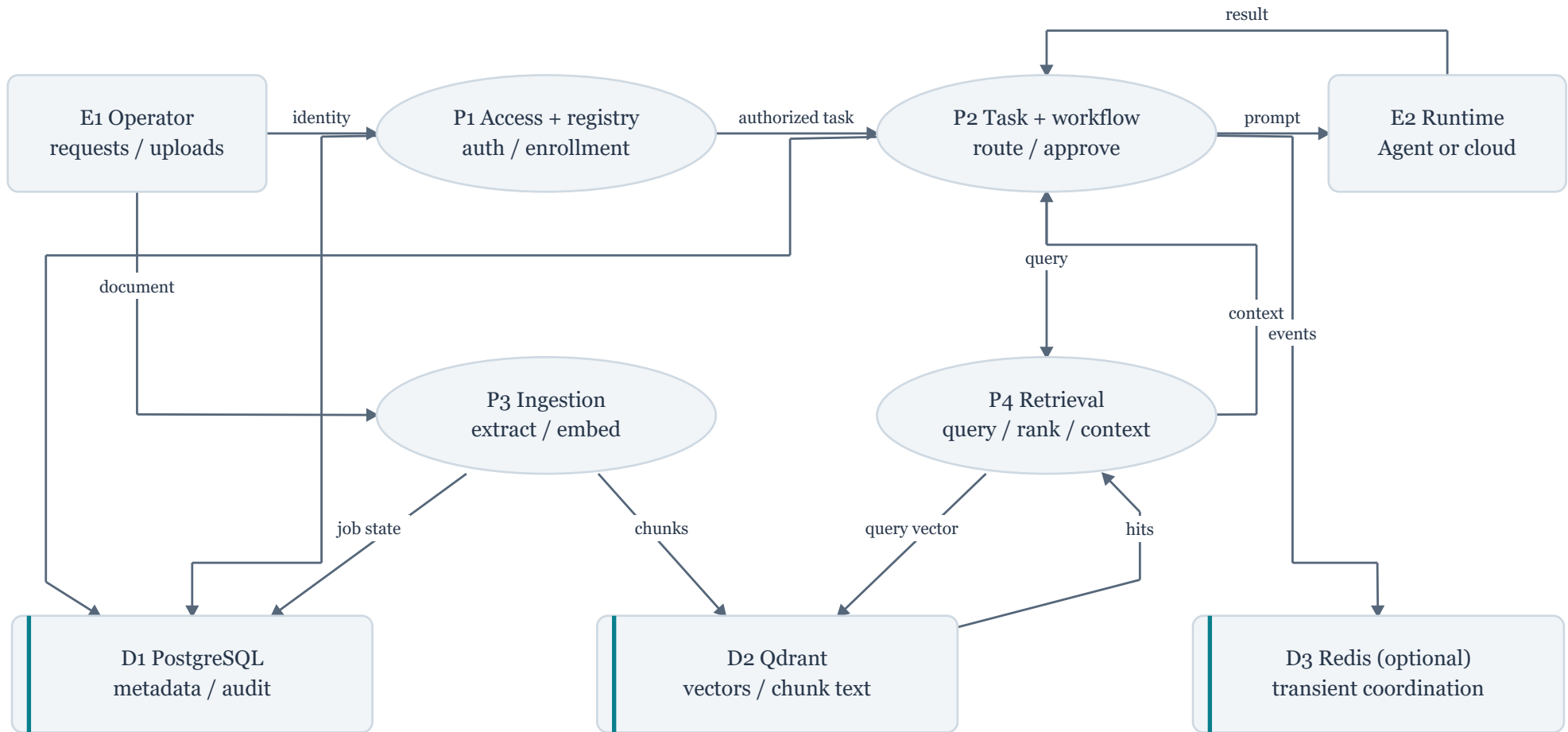


Figure 2. Level-1 information movement

Processes P1-P4 transform requests into execution and knowledge results. D1 is relational metadata; D2 is vector content; D3 is transient coordination, not a queue of durable jobs. Uploaded text becomes chunks and vector payloads; task outputs and workflow state are persisted separately. The external inference boundary is crossed only for an explicitly selected cloud route. [L2, L6]

09 Data governance and retrieval semantics

9.1 Data classification and minimization

Task prompts, retrieved passages and model outputs can contain confidential information. Provider keys are a separate class of secret: they belong in the configured vault or explicitly selected process-memory store, never in workflow definitions or report artifacts. Audit records identify operations and actors, but auditability does not justify retaining raw secrets. Retention periods and user-level knowledge isolation need deployment policy; role-based authorization alone does not demonstrate tenant isolation. [L2]

9.2 Retrieval is not truth verification

Qdrant stores searchable vectors with payloads (Qdrant, n.d.); similarity is a retrieval signal, not a guarantee of relevance or truth. The controller supports named-collection scope and result limits. The Agent's new evidence contract sorts, filters, deduplicates and truncates supplied chunks, then maps source labels to provenance. Citation status validates recognized identifiers only. Prompt injection within evidence remains a residual risk because instruction separation and refusal normalization cannot prove semantic compliance. [L2, L3]

9.3 Important asymmetry

Cloud tasks fail closed if retrieval raises an error. The legacy local controller route instead logs the retrieval failure and may dispatch without context. Successful retrieval with no relevant hits returns a refusal without invoking a model. These are three distinct outcomes and should be visible to an operator. The stronger Agent evidence API does not automatically change this legacy behavior; migration must be intentional and regression-tested. [L2, L3]

10 Use case diagram

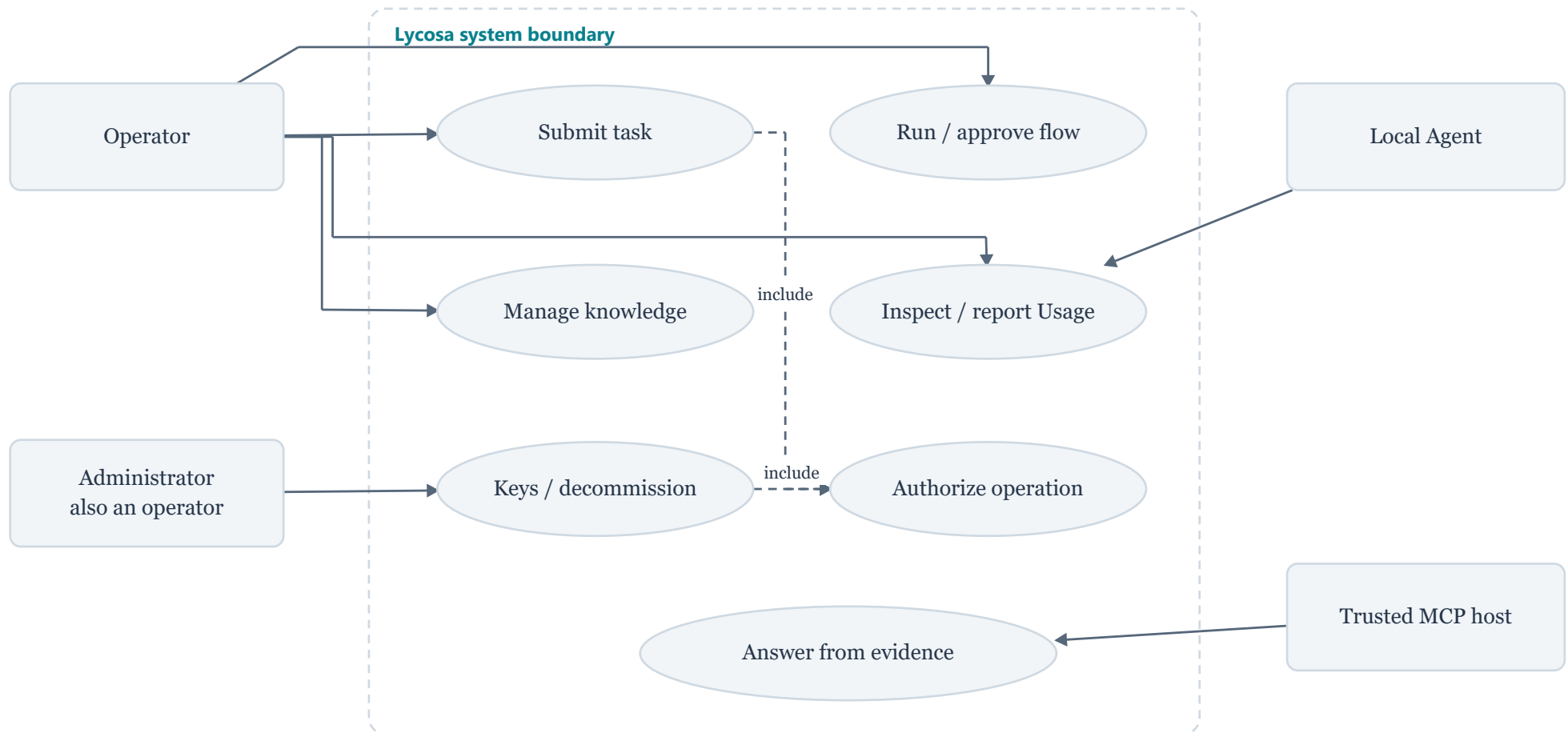


Figure 3. Actor goals and system boundary

The administrator also performs operator activities. The node Agent is a supporting software actor, not a human role. The MCP host is an optional external client of the local companion. Dashed inclusion links identify a required sub-function in this analytical view; authorization is a required operation-level check. No actor receives unrestricted shell access through these use cases. [L2-L4]

10 Use case contracts and exceptions

10.1 UC-T: submit a task

Precondition: an authenticated principal has task-submission permission. The main flow validates the payload, persists the task, applies privacy and retrieval rules, selects the execution route and records a terminal result. Local dispatch may create multiple TaskExecution attempts. Alternative flows include no eligible node, absent model, provider credential failure, transport timeout and empty retrieved evidence. Postcondition: a terminal result or sanitized error is inspectable; a transport-level success is not treated as proof of answer quality. [L2, L5]

10.2 UC-W: approve a workflow

A workflow pauses at an approval step after earlier outputs are persisted. The reviewer inspects the run and submits an explicit decision. Approval resumes execution; rejection prevents downstream work. Conditional steps can be skipped and retries create additional attempt records. An approval gate is not a universal safeguard if a definition omits it, and review after execution cannot retroactively protect data already sent to an external provider. Place the gate before the consequential action. [L2, L5]

10.3 UC-M: answer from supplied evidence

The caller supplies an installed-model identifier, question and bounded evidence. Empty evidence yields a refusal without generation. Unknown models fail before inference; excess parallel demand is rejected rather than queued indefinitely. A completed response contains output, source map, citation status and truncation metadata. The caller remains responsible for deciding whether missing or invalid citations are acceptable. The MCP companion exposes this narrow function without file browsing or external server discovery. [L3]

11 System class diagram

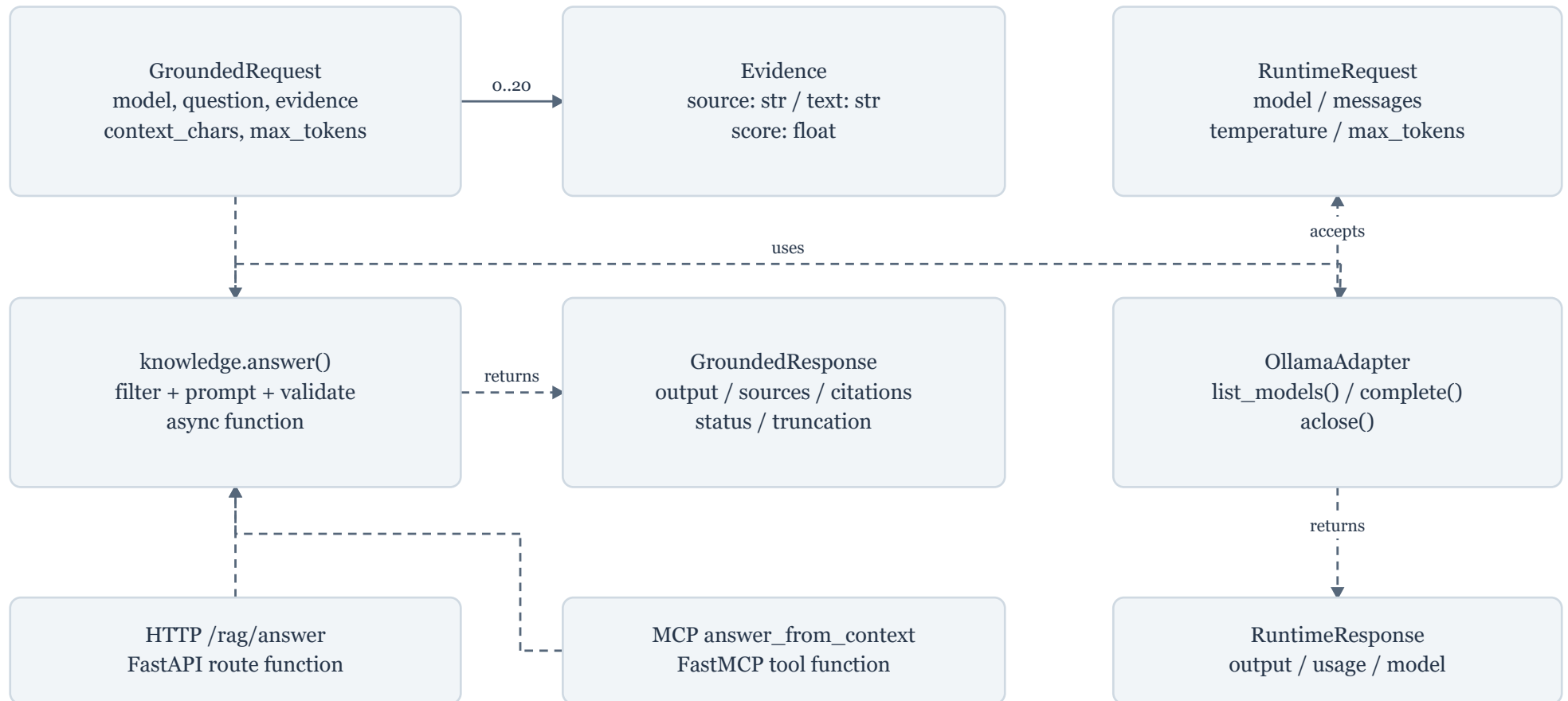


Figure 4. Runtime and evidence contracts

This UML-style view focuses on concrete Python contracts in the Agent, not every application class. Composition denotes nested request/response values; dashed arrows show function dependencies rather than inheritance. FastMCP and the authenticated FastAPI endpoint both invoke the same evidence-answer function. The HTTP route and stdio tool have independent per-process concurrency controls. [L3]

11 Interface constraints and resource budgets

11.1 Bounded evidence contract

GroundedRequest permits at most 20 evidence chunks. Each chunk has a source, text and finite score; the aggregate selected context is bounded to at most 32,000 characters and generated output to 4,096 tokens. Installed-model discovery has a five-second deadline and generation a ninety-second deadline. The two-slot admission bound limits this new evidence path, not all inference on the machine. Character limits do not guarantee that a model's token context will fit. [L3]

11.2 Validation and abstention

Pydantic rejects unknown fields and invalid bounds. Sources receive S1, S2 and subsequent labels after ordering and deduplication. Citation extraction reports recognized and invalid IDs without manufacturing missing citations. English outputs beginning with selected explicit abstention phrases are normalized to a canonical refusal and stripped of unsupported elaboration. This conservative heuristic is intentionally narrower than multilingual semantic verification and can suppress a partially useful answer; evaluation should include such false-abstention cases. [L3, L5]

11.3 Evolution strategy

A future version should negotiate runtime capability and context limits explicitly, report structured refusal reasons and validate evidence sufficiency independently of generation. It should preserve the existing narrow authority boundary until sandboxed tools, consent and auditability are designed together. Extending the MCP tool list is not by itself a safe autonomous-agent architecture. Contract versioning and compatibility tests should precede a controller-wide switch to the structured evidence API.

12 System interaction diagram

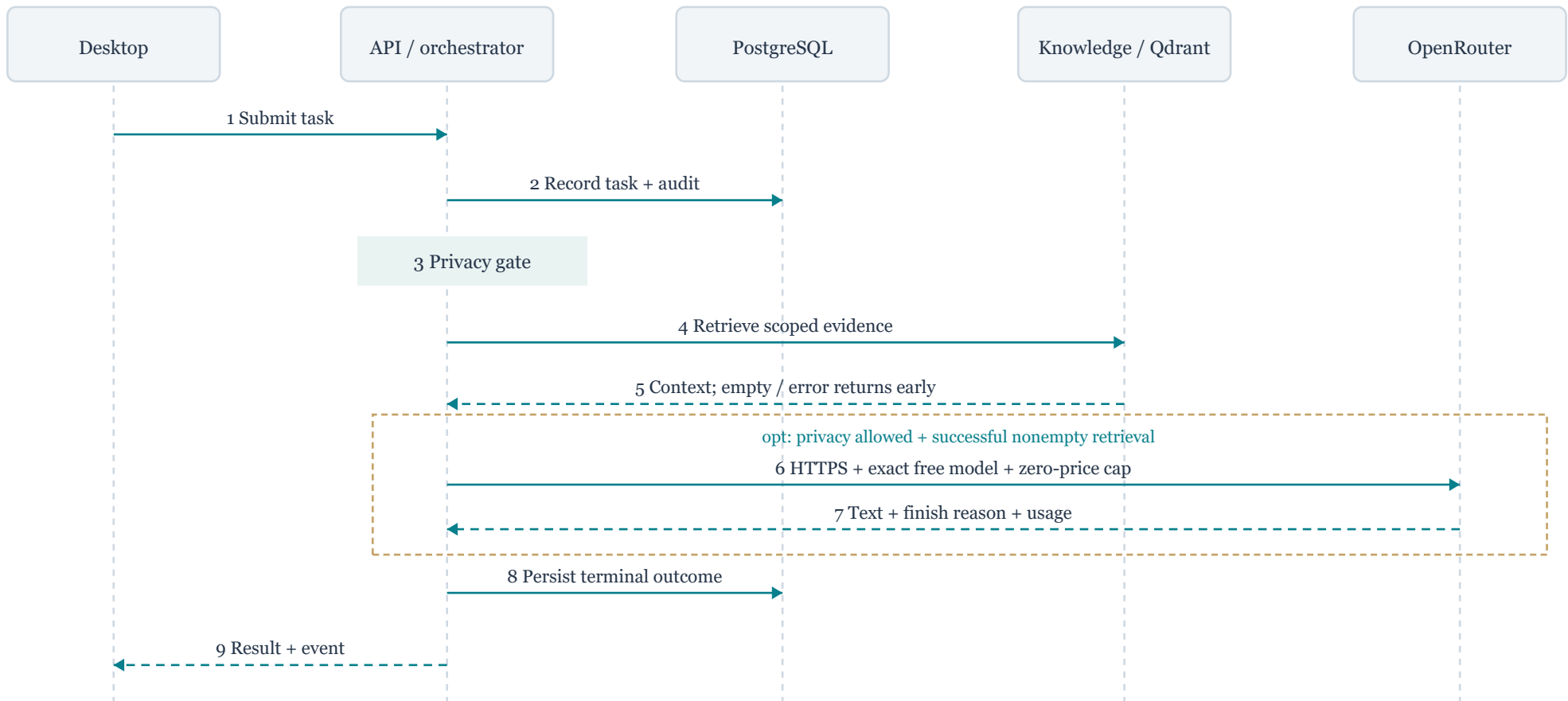


Figure 5. Knowledge-enabled OpenRouter task

The sequence shows the direct controller cloud path, not local Agent execution. Privacy denial occurs before retrieval and external dispatch. Retrieval failure cancels cloud execution; no relevant evidence produces a refusal. Only an eligible request reaches the provider. PostgreSQL stores the task outcome; dashboard events are notifications and the persisted task remains the record of result. [L2]

12 Execution semantics and failure handling

12.1 Synchronous orchestration

The API synchronously awaits task execution and returns a completed task; dispatch is shielded against client disconnect, not controller process failure. Workflow requests run until completion or an approval pause. This is simpler to operate than a separate job system but ties latency, disconnects and process lifecycle to execution. Recording an attempt before network dispatch improves post-failure diagnosis. It does not provide exactly-once inference, automatic recovery of every interrupted workflow, or durable delivery of all live events. [L2]

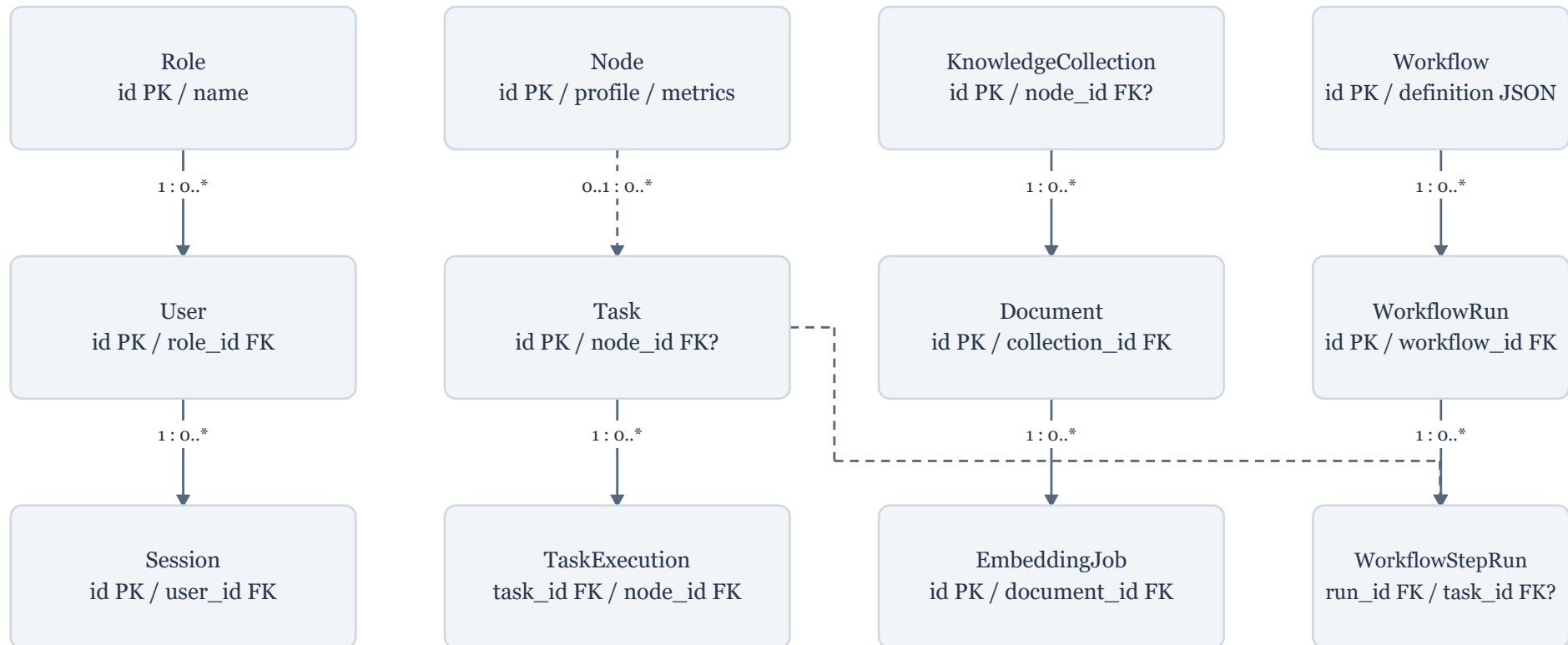
12.2 Cloud-specific safeguards

The OpenRouter adapter uses a fixed HTTPS origin, refuses redirects, validates the exact model and sets a zero-price provider cap. It rejects privacy-required requests and redacts the active credential from retained text. A length finish reason is treated as a truncated-response failure in the reviewed source. The guard was added after a low-token test produced only 1 for 7 + 8. The later larger-budget test produced 15. Neither event establishes broad model competence. [L2, L5; OpenRouter, n.d.]

12.3 Operational implications

A successful source change is not active until its process is rebuilt or restarted. Session credentials then need deliberate reconfiguration. Production rollout should identify affected services, verify database compatibility, preserve previous images and perform health, authorization and synthetic task checks. The report does not silently equate a pushed commit with a deployed release. Model availability and external quotas must be rechecked at deployment time.

13 Entity-relationship diagram



FK? = nullable; selected core entities

Figure 6. Core relational schema

Every displayed table uses a UUID primary key. Links are selected foreign keys, with 1:N labels indicating one parent to zero or more children. Dashed annotations identify nullable associations or deliberately omitted cross-cutting links. Qdrant points are outside the relational schema; their document identifiers are application-level references, not PostgreSQL foreign keys. [L2]

13 Persistence, cardinality and integrity

13.1 Three lifecycle aggregates

Task owns zero or more execution attempts and may reference a winning node. A direct OpenRouter task has no node execution row, so a successful task with a null node is valid. Workflow owns runs, and each run owns step-attempt records; a task-kind step may reference a Task. KnowledgeCollection owns Documents, which own EmbeddingJobs. RetrievalRequest records scope, counts and latency rather than every generated answer. [L2]

13.2 Identity and cross-cutting references

Role relates to User and ApiKey; User owns Session. Nullable actor fields on tasks, retrievals and audit logs distinguish user and API-key initiation. ApiKey may be bound to a Node. AuditLog also stores a resource type and identifier, which are not a universal polymorphic foreign key. The diagram omits some repeated actor links to remain legible; this is an intentional projection, not a complete migration script. Node-Agent-AgentCapability tables exist, but current registration advertises capabilities in the Node hardware-profile JSON rather than populating those capability rows. [L2, L3]

13.3 Consistency responsibilities

SQL transactions do not atomically include Qdrant or remote inference. Ingestion and deletion therefore need explicit failure handling, idempotent operations and reconciliation. A relational document marked embedded is meaningful only if its vectors remain available. Recovery procedures should restore database and vector snapshots coherently and test retrieval afterward. Schema uniqueness, foreign keys and audit history are useful controls, but they do not replace restore drills, retention policy or authorization tests.

Appendix A | Verification and risks

A.1 Evidence ledger

Fresh report-date regression evidence: 311 backend tests, 48 Agent tests and 4 targeted Flutter provider-control tests passed. The earlier complete Flutter suite recorded 76 passing tests and was not fully rerun for this report. These are finite test suites, not field reliability estimates. The historical live MCP test returned a supported code with [S1], and an unsupported question returned the canonical refusal after normalization. Desktop inspection showed the successful NVIDIA result, populated provider selection, workflow list and knowledge collections; local nodes were offline at that observation. [L5]

A.2 Risk register

R1: concurrent placement can overcommit RAM/VRAM; introduce reservations and contention tests. R2: request-bound execution can be interrupted; design durable admission and restart recovery. R3: prompt injection or misleading citations can survive syntactic checks; evaluate adversarial and insufficient-evidence corpora. R4: default embeddings disadvantage Arabic; select and measure multilingual retrieval before claiming support. R5: external quotas/model changes can break cloud execution; retain explicit failure and no-paid-fallback policy. R6: credentials or prompts can cross an unprotected deployment link; require reviewed transport configuration. [L2-L6]

A.3 Research validity

No participant study, statistical comparison, penetration assessment or production-scale endurance test is claimed. Synthetic arithmetic and support-code fixtures are useful integration probes but are weak samples of real reasoning. Future evaluation should pre-register workloads, hardware, baselines and scoring rules; report failures and confidence intervals rather than only selected successes. This risk-oriented evaluation stance is consistent with NIST's emphasis on managing AI risks across the lifecycle, without asserting formal compliance (NIST, 2023).

Appendix B | Conclusions and roadmap

B.1 Engineering conclusion

Lycosa demonstrates a practical integration of device inventory, explicit routing, knowledge retrieval and approval-aware workflows. Its strongest design property is separation of responsibility: operational state belongs to the controller, inference belongs to a selected runtime, and provider authority is explicit. Its principal maturity gap is that durable state and bounded interfaces do not by themselves guarantee correct answers or resilient distributed execution. The defensible outcome is a tested engineering increment, not a universal autonomous AI infrastructure. [L1-L6]

B.2 Prioritized engineering roadmap

Priority 1 is operational correctness: deploy the reviewed fixes, document credential reconfiguration, test restoration and expose incomplete output clearly. Priority 2 is grounding quality: unify structured evidence contracts, measure Arabic/English retrieval, add reranking only if it improves declared metrics and evaluate abstention precision. Priority 3 is controlled extensibility: provider-neutral adapters, narrowly permissioned MCP clients and sandboxed tools with human approval. OpenAI, Gemini and DeepSeek should remain labelled planned direct integrations until contracts, secret handling and acceptance are implemented.

B.3 Reproducibility and release discipline

Run backend tests from backend/ so local production environment settings do not leak into the test configuration. Agent tests include a real stdio subprocess but mock most inference; live smoke scripts are deliberately opt-in. The PDF builder retains parallel language content and vector diagram definitions in reports/. A release should record the commit, dependency lock state, service image and configuration without secrets. Keep source, test evidence, installed build and deployment status separately traceable.

References | Project evidence

L1 abdra7 (2026). Lycosa repository, README and Sprint 12/OpenRouter engineering notes. Inspected 18 September 2026.

<https://github.com/abdra7/Lycosa>

README.md; docs/SPRINT12.md; docs/OPENROUTER.md

L2 abdra7 (2026). Controller implementation: API, orchestration, scheduling, knowledge, workflow and relational models.

<https://github.com/abdra7/Lycosa/tree/main/backend/app>

backend/app/services/; backend/app/models/; backend/app/api/v1/

L3 abdra7 (2026). Agent runtime, bounded knowledge answering and optional MCP server.

<https://github.com/abdra7/Lycosa/tree/main/agent>

agent/lycosa_agent/knowledge.py; executor.py; mcp_server.py; runtimes/

L4 abdra7 (2026). Flutter desktop dashboard and provider controls.

<https://github.com/abdra7/Lycosa/tree/main/dashboard/lib>

dashboard/lib/core/api_client.dart; features/tasks/; features/admin/

L5 abdra7 (2026). Automated test suites and opt-in acceptance scripts. Historical live evidence dated 7 September 2026 is distinguished in the text.

<https://github.com/abdra7/Lycosa>

backend/tests/; agent/tests/; dashboard/test/; backend/scripts/; agent/scripts/

L6 abdra7 (2026). Compose deployment and coordination configuration.

<https://github.com/abdra7/Lycosa/tree/main/infra>

infra/docker-compose.yml; backend/app/core/events.py; window_store.py

References | External scholarship

R1 Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 33. arXiv:2005.11401.

<https://arxiv.org/abs/2005.11401>

Accessed 18 September 2026

R2 Model Context Protocol (2025). Architecture, specification dated 25 November 2025.

<https://modelcontextprotocol.io/specification/2025-11-25/architecture>

Accessed 18 September 2026

R3 FastAPI (n.d.). Deployments Concepts. Official technical documentation.

<https://fastapi.tiangolo.com/deployment/concepts/>

Accessed 18 September 2026

R4 Qdrant (n.d.). Collections. Official technical documentation.

<https://qdrant.tech/documentation/manage-data/collections/>

Accessed 18 September 2026

R5 OpenRouter (n.d.). API Reference. Official request and response specification.

https://openrouter.ai/docs/api_reference/overview

Accessed 18 September 2026

R6 NIST (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0).

<https://www.nist.gov/itl/ai-risk-management-framework>

Accessed 18 September 2026